

The guide has two halves because the two CREST exams use nmap for different jobs:

- Everything below runs on the lab analyst boxes (`cpsa-analyst` :8031-8036, `cpia-forensics` :8021-8025) against the lab targets. Never point nmap at a system you do not own or have written permission to scan.

One tool, three jobs:

J	o	b							
C	P	S	A		u	s	a	g	e
C	P	I	A		u	s	a	g	e
Discover	Which hosts are alive on the range?	What did the attacker see?							
Enumerate	What ports and services are exposed?	Which of those were actually hit?							
Assess	What versions / known weaknesses?	Does the evidence match the exposure?							

Version banner: `nmap --version` should print `Nmap version 7.x`.

2. Getting your bearings

Open the terminal on the analyst box (CPSA: the **01 Network discovery** card, port 8031; CPIA: the **Packet capture triage** card, port 8021) and check you are in the right place:

```
whoami      # should be root on the analyst boxes
hostname    # cpsa-analyst or cpia-forensics
ip a        # show interfaces and addresses
```

You should see the analyst box's own address on the lab network. The CPSA target is `cpsa-target`; the CPIA evidence servers are the lab services.

Likely errors and fixes:

- `nmap: command not found` — the toolset is missing. On the analyst box run `apt-get update && apt-get install -y nmap` (the lab images normally ship it; this is a fallback for a base image).
 - `ip: command not found` — use `ifconfig` or `hostname -I` instead.
-

3. Target specification — telling nmap what to scan

Nmap accepts one or many targets, in several forms, and can mix them:

```
nmap 10.0.0.1          # a single IP
nmap 10.0.0.1 10.0.0.2  # several IPs (space-separated)
nmap 10.0.0.1-20        # a range (dash notation)
nmap 10.0.0.0/24        # a CIDR block
nmap -iL targets.txt    # read targets from a file, one per line
nmap -iL targets.txt -sn # ... handy for big discovery jobs
nmap 10.0.0.1 --exclude 10.0.0.5 # skip specific hosts
nmap 10.0.0.0/24 --exclude-file skip # skip hosts from a file
```

Rules of thumb:

- **CIDR is the lab default.** `10.0.0.0/24` = 254 usable addresses. `/16` is 65,534 — know the maths: each bit shaved off the mask doubles the size.
 - **Quoting matters in scripts:** `nmap 10.0.0.1-20` is fine, but if you ever build targets in a variable, quote it: `nmap "$TARGETS"`.
 - **DNS lookups can slow you down;** use `-n` to skip reverse DNS and `-R` to force it. In the lab, `-n` is usually right.
-

4. Host discovery — finding what is alive (CPSA step 1)

Before port-scanning, find the hosts. The default discovery probe is a combination of an ICMP echo request, a TCP SYN to port 443, a TCP ACK to port 80, and an ICMP timestamp request:

```
nmap -sn 10.0.0.0/24      # ping sweep: which hosts answer?
nmap -sn -PS 10.0.0.0/24  # TCP SYN ping to port 80 (443 with -PS443)
nmap -sn -PA 10.0.0.0/24  # TCP ACK ping
nmap -sn -PU 10.0.0.0/24  # UDP ping (port 40125 by default)
nmap -sn -PE 10.0.0.0/24  # ICMP echo only
nmap -sn -PP 10.0.0.0/24  # ICMP timestamp only
```

What each does:

P	r	o	b	e	
P	a	c	k	e	t
N	o	t	e	s	
-PE	ICMP echo request	The classic ping; often blocked			
-PS80	TCP SYN to 80	Sends SYN, treats SYN/ACK or RST as alive			
-PA80	TCP ACK to 80	Firewalls that drop SYN but pass ACK still answer			
-PU	UDP probe	Useful when TCP+ICMP are filtered			
-Pn	none	Skip discovery: assume everything is up			

-Pn is the workhorse of the lab — many lab services and hardened hosts

drop the discovery probes, but still answer port scans:

```
nmap -Pn 10.0.0.20      # don't ping; just scan ports
nmap -Pn -sn 10.0.0.0/24 # discovery without ping: slow, be patient
```

Likely errors and fixes:

- The sweep shows *all* hosts down but port scans find open ports — you are behind a host that drops ICMP. Add **-Pn** and move on.
- Sweep is extremely slow — add **-T4** (see Section 9) or **--min-rate 1000**.

5. Port scanning — the core technique

5.1 Which ports, and how

```
nmap -Pn 10.0.0.20      # default: top 1000 TCP ports
nmap -Pn -p 22 10.0.0.20 # one port
```

```

nmap -Pn -p 22,80,443 10.0.0.20      # several
nmap -Pn -p 1-1024 10.0.0.20        # a range
nmap -Pn -p- 10.0.0.20              # ALL 65535 TCP ports (slow but complete)
nmap -Pn --top-ports 200 10.0.0.20  # most-common 200
nmap -Pn -p T:22,80,U:53 10.0.0.20  # mix TCP and UDP
nmap -Pn -r 10.0.0.20              # scan ports in numeric order, not random
nmap -Pn --exclude-ports 22 10.0.0.20 # skip specific ports

```

In the CPSA lab the starter command is **nmap -sV -p- cpsa-target** — the

full port range plus version detection. On a Wi-Fi lab link that takes a few

minutes; **-p-** is exhaustive and the exam rewards it, but always pair it with a

fast **--top-ports** pass first so you have results while the big scan runs.

5.2 Scan techniques (the **-s** flags)

F	l	a	g								
N	a	m	e								
H	o	w		i	t		w	o	r	k	s
R	o	o	t	?							
N	o	t	e	s							
- sT	TCP connect	Full TCP handshake via the OS	no	Slow, but works unprivileged; shows up in logs							
- sS	SYN "half-open"	SYN, then RST — never completes	yes	Default as root; faster, stealthier							
- sU	UDP scan	UDP probes per port	yes	Slow; ICMP port-unreachable = closed							
- sA	ACK scan	Maps firewall rules, not open ports	yes	Can't tell open from filtered							
- sW	Window scan	TCP window-size oddity	yes	Rarely needed in the lab							
- sN/ - sF/ - sX	Null/Fin/Xmas	Flag tricks to bypass simple filters	yes	Mostly historical; useful for detection study (CPIA)							

The state machine every analyst must be fluent in:

```

open      -> service responded (SYN/ACK on -sS)
closed    -> RST received (nothing listening)
filtered  -> no response or ICMP unreachable (a firewall is eating packets)
unfiltered -> reachable, but state unknown (ACK scan)
open|filtered -> can't distinguish (UDP, null, fin)

```

Example of reading states:

```
nmap -Pn -p 22,80,443,8080 10.0.0.20
# PORT      STATE      SERVICE
# 22/tcp    open       ssh
# 80/tcp    open       http
# 443/tcp   filtered   https
# 8080/tcp   closed     http-proxy
```

Likely errors and fixes:

- Everything shows `filtered` — you scanned the wrong host, the target is firewalled, or you scanned with UDP flags that the target drops. Check the IP, try `-sT`.
- `-sS` says "requires root privileges" — you are not root on the box. Use `-sT`, or `sudo` if available.
- `-p-` appears to hang — it is not hanging, it is working through 65k ports. Watch it with `--stats-every 30s`, or interrupt and switch to `--top-ports`.

6. Service and version detection — identifying what is listening (CPSA step 2)

Open ports are only useful when you know what is behind them:

```
nmap -Pn -sV 10.0.0.20          # version detect on open ports
nmap -Pn -sV --version-intensity 9 # max probes (0-9, default 7)
nmap -Pn -sV --version-light      # intensity 2: fast, fewer probes
nmap -Pn -sV -sC 10.0.0.20        # version + default scripts
nmap -Pn -sV -p 22,80,443 10.0.0.20 # version on selected ports
```

`-sV` sends probes and reads banners; the output adds a `VERSION` column:

```
22/tcp    open  ssh      OpenSSH 9.6p1 Debian 6 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.62 ((Debian))
3306/tcp  open  mysql    MySQL 8.0.37
```

Why the CPSA exam cares: **the version is the vulnerability.** `Apache 2.4.62`

tells you which CVEs apply; `OpenSSH 9.6p1` tells you whether the target is patched. The CPIA exam cares for the opposite reason: banners in evidence (pcap, logs) must be consistent with what the scan reports.

Likely errors and fixes:

- `VERSION` column shows nothing — the service did not answer probes; rerun with `--version-intensity 9` or check the port is really open.

- Version detection slows the scan massively — limit `-sV` to the open ports you actually care about, or use `--version-light`.

7. Operating system detection (CPSA step 3)

```
nmap -Pn -O 10.0.0.20          # OS fingerprinting (needs root)
nmap -Pn -O --osscan-guess 10.0.0.20 # guess harder when the match is weak
nmap -Pn -O --max-os-tries 1      # limit tries (faster)
```

`-O` sends a stack of TCP/IP probes and matches the responses against a fingerprint database:

```
OS detection performed. Please report any incorrect results.
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel
OS details: Linux 4.15 - 5.6, Linux 5.0 - 5.4
```

Notes:

- OS detection **requires root** (it builds raw packets).
- On Wi-Fi or behind NAT the fingerprint degrades. If the output says "OSScan results may be unreliable", say so in your report rather than guessing — the exam marks honesty, not confidence.
- The **CPE** (Common Platform Enumeration) string is the important bit for reporting: `cpe:/o:linux:linux_kernel` is machine-readable.

Likely errors and fixes:

- "requires root privileges" — run as root on the analyst box (you normally are) or use `sudo nmap`.
- "No OS matches" — one open + one closed port is needed for a good match.

Run a fuller port scan first, then `-O` on the results.

8. Output formats — evidence you can hand in

Every exam and every report wants reproducible evidence:

```
nmap -Pn -sV -oN scan.txt 10.0.0.20    # normal text
nmap -Pn -sV -oX scan.xml 10.0.0.20    # XML (machine-readable)
nmap -Pn -sV -oG scan.grep 10.0.0.20   # grepable
nmap -Pn -sV -oA lab1 10.0.0.20        # all three at once (best)
nmap -Pn --open -oN open.txt 10.0.0.20 # only show open ports
nmap -Pn -sV -oN s.txt -oX s.xml 10.0.0.20 # multiple formats
```

The `-oA lab1` prefix produces `lab1.nmap`, `lab1.xml` and `lab1.gnmap` — the `.nmap` file is the human-readable one:

```
cat lab1.nmap
# Nmap 7.94 scan initiated ...
# Host 10.0.0.20 (...) is up
# PORT      STATE SERVICE VERSION
# 22/tcp    open  ssh      OpenSSH 9.6p1 Debian 6 (protocol 2.0)
```

XML is greppable for scripting:

```
grep 'portid="22"' lab1.xml
```

Likely errors and fixes:

- File written to the wrong directory — nmap writes where you ran it. Use an absolute path: `-oA /work/nmap/lab1`.
- `-oN` overwrites silently. Use `--append-output` or a fresh filename per run.

9. Timing and performance — controlling speed and noise

```
nmap -T0 10.0.0.20    # paranoid: insanely slow, one probe at a time
nmap -T1 10.0.0.20    # sneaky
nmap -T2 10.0.0.20    # polite
nmap -T3 10.0.0.20    # normal (default)
nmap -T4 10.0.0.20    # aggressive — the lab default choice
nmap -T5 10.0.0.20    # insane: may drop responses
nmap --min-rate 1000 10.0.0.20 # at least 1000 packets/sec
nmap --max-rate 100  10.0.0.20 # never more than 100/sec (quiet)
nmap --host-timeout 5m 10.0.0.20 # give up on a host after 5 minutes
nmap --max-retries 1 10.0.0.20  # don't resend probes forever
nmap --stats-every 30s 10.0.0.20 # progress while you wait
```

Guidance:

- **In the lab, `-T4` is your friend** — fast and reliable on a local link.
 - **For detection study (CPIA), use `-T0` / `-T1`** and watch how slowly a careful attacker moves. Real attackers tune for stealth; your detections must catch slow scans too.
 - `--min-rate` beats `-T5` when you need a *guaranteed* throughput.
 - Long scans: always add `--stats-every` so you can see progress, and consider running the scan in the background (`nmap ... &` or a tmux session).
-

10. The Nmap Scripting Engine (NSE) — automation and enumeration

NSE scripts extend nmap into an enumeration and assessment platform:

```
nmap -Pn -sC 10.0.0.20          # default script set (-sC)
nmap -Pn -sC -sV 10.0.0.20     # the classic combo
nmap -Pn --script http-title 10.0.0.20 # one script
nmap -Pn --script "http-*" 10.0.0.20  # wildcard category
nmap -Pn --script smb-enum-shares 10.0.0.20
nmap -Pn --script vulners --script-args mincvss=7 10.0.0.20 # CVE lookup
nmap -Pn --script-args http-title.timeout=10 ...
```

Script categories you will actually use:

C	a	t	e	g	o	r	y												
W	h	a	t		i	t		c	h	e	c	k	s						
E	x	a	m																
safe	No disruptive actions	both																	
default	What -sC runs	both																	
discovery	Service/version info, hostnames	CPSA																	
auth	Credentials / auth bypass attempts	CPSA																	
brute	Password guessing — loud, slow	CPSA (know it, don't run it in the lab)																	
exploit	Exploit attempts — never run in the lab	neither																	
vuln	Known-vulnerability checks	CPSA																	
malware	Malware/backdoor checks	CPIA																	

Useful scripts to memorise:

```
nmap --script=http-title http://10.0.0.20          # page <title> of the site
nmap --script=http-enum 10.0.0.20                  # common web paths/dirs
nmap --script=smb-os-discovery 10.0.0.20           # SMB hostname/OS/domain
nmap --script=smb-enum-shares 10.0.0.20            # SMB shares (often reveals data)
nmap --script=ftp-anon 10.0.0.20                   # anonymous FTP login
nmap --script=ssh2-enum-algos 10.0.0.20            # SSH key-exchange algorithms
nmap --script=dns-zone-transfer -p 53 ns.example.com # zone transfer attempt
```

Likely errors and fixes:

- Scripts take forever — NSE waits on timeouts; add `--script-timeout 30s`.
- `brute / exploit` categories must stay **off** in the lab: brute-forcing is

loud and the exploit scripts may break the lab services. In an exam, running

`exploit` scripts without permission is a fail.

11. Firewall and IDS evasion — the detection half (CPIA-heavy)

These options matter for two audiences: attackers who want to be quiet, and

analysts who must recognise the techniques in evidence.

```
nmap -f 10.0.0.20 # fragment packets (8 bytes)
nmap -f -f 10.0.0.20 # fragment into 16-byte pieces
nmap -D 10.0.0.5,10.0.0.6,ME 10.0.0.20 # decoy source addresses
nmap -D RND:5 10.0.0.20 # 5 random decoys
nmap -g 53 10.0.0.20 # source port 53 (DNS – often trusted)
nmap --source-port 53 10.0.0.20 # same thing, explicit
nmap --spooof-mac 00:11:22:33:44:55 10.0.0.20 # spoof MAC (same subnet)
nmap --data-length 32 10.0.0.20 # pad packets to a fixed size
nmap --ttl 64 10.0.0.20 # set IP TTL
nmap --badsum 10.0.0.20 # send bad checksums (detects dumb firewalls)
nmap --proxies http://10.0.0.1:8080 10.0.0.20 # route through a proxy chain
```

What each one is *for*, and how to *detect* it:

T	e	c	h	n	i	q	u	e																
A	t	t	a	c	k	e	r		i	n	t	e	n	t										
H	o	w		y	o	u		s	e	e		i	t		a	s		a	n		a	n		
- f / fragmentation	evade simple filters that can't reassemble	IDS reassembles fragments; watch for tiny/fragmented packets in pcap																						
- D decoys	hide the real source IP	correlation: one scan, many source IPs, same pattern																						
- g 53	ride the DNS port	rule: never trust source port alone																						

	past naive rules	
--spoof-mac	hide the NIC identity	ARP table vs MAC of record
--data-length	normalise packet size	packet-size heuristics
-T0/-T1	slow scans to dodge rate-based alerts	time-window correlation — a port sweep spread over hours still lands in logs

The CPSA exam wants you to *understand* these; the CPIA exam wants you to *detect* them. Both expect the professional answer: evasion exists, and detecting a scan is only the start — you then decide what it means for the incident.

12. The CPSA workflow — assessment end to end

This is the canonical sequence for the CPSA labs (01-03):

```
# STEP 1 – discovery: what is alive?
nmap -sn -T4 10.0.0.0/24 -oN /work/recon-discover.txt

# STEP 2 – fast top-ports pass for a quick map
nmap -Pn -T4 --top-ports 200 -oN /work/recon-fast.txt 10.0.0.20

# STEP 3 – full-range scan with versions (the lab's starter command)
nmap -Pn -sV -p- -T4 -oA /work/lab1 10.0.0.20

# STEP 4 – OS fingerprint (root)
nmap -Pn -O --osscan-guess -oN /work/lab1-os.txt 10.0.0.20

# STEP 5 – targeted scripts on the services you found
nmap -Pn -sV -p 80,443 --script "http-title,http-enum" -oN /work/lab1-web.txt 10.0.0.20
nmap -Pn -p 445 --script "smb-os-discovery,smb-enum-shares" -oN /work/lab1-smb.txt 10.0.0.20

# STEP 6 – consolidate everything into the worksheet
cat /work/lab1.nmap /work/lab1-web.txt /work/lab1-smb.txt
```

The worksheet table you should be able to fill from your scans:

the lab target and compare: `nmap -sS -p- -oN /work/attacker-view.txt` .

- **What did they do with it?** The post-scan traffic (banner grabs, SMB calls, HTTP requests) shows which open ports they acted on.

13.2 Proving exposure during reconstruction

During live-incident reconstruction you often need to verify "was this service actually reachable at the time?" Nmap gives you the current state — pair it with the historical evidence:

```
nmap -Pn -sV -p 445 cpia-victim      # is SMB still exposed now?
grep -c "445" /var/log/auth.log      # did anything connect to it?
```

If the pcap shows an SMB handshake and your scan confirms port 445 open, the evidence chain is complete. If the scan disagrees with the pcap, note the discrepancy — that is a finding, not a mistake.

13.3 Safe-scanning rules for the forensic lab

- **Never run exploit or brute scripts.** The lab services are evidence, not targets.
- **Scan only the lab range.** Your IP discipline is part of the marks.
- **Record every scan** with `-oA` so your analysis is reproducible — an examiner will ask "what exactly did you scan, and when?"

14. Common errors and fixes — the master list

S	y	m	p	t	o	m
C	a	u	s	e		
F	i	x				
Failed to determine route to	No route to the target network	ip route ; check you're on the lab network				
Everything filtered	Firewall dropping probes, or wrong IP	-Pn -sT , verify target address				
-sS needs root	Non-root shell	sudo nmap or use -sT				
-O needs root	Same	sudo nmap -O				
Scan "hangs"	Big range, slow timing	-T4, --min-rate, --stats-every 30s				

Version detection missing	Service not answering probes	<code>--version-intensity 9</code> , or <code>--version-light + -sC</code>
Host shown down but ports open	ICMP dropped	<code>-Pn</code>
Couldn't determine your gateway	Interface/DNS config on the box	<code>ip a</code> , check the lab compose network
NSE scripts timeout	Long-running scripts	<code>--script-timeout 30s</code>
Output file "not found"	Wrong working directory	Use absolute <code>-oA /work/...</code> paths
PORT STATE SERVICE but no VERSION	<code>-sV</code> not used	Add <code>-sV</code>

15. Cheat sheet — every flag in this guide

```
# TARGETS
-iL <file>          targets from a file      --exclude <ip>      skip hosts
10.0.0.0/24         CIDR block                  -n / -R             no/force DNS

# DISCOVERY
-sn                ping sweep only              -Pn                 skip discovery
-PS/PA/PU/PE/PP    SYN/ACK/UDP/echo/timestamp probes

# PORT SCANS
-sS SYN (root, default)  -sT connect (no root)  -sU UDP
-p- all 65535           -p 22,80,443           --top-ports N
-r numeric order        --exclude-ports

# SERVICE + OS
-sV versions (--version-intensity 0-9)  -O OS fingerprint
--osscan-guess force weak matches

# SCRIPTS
-sC default set          --script <name|category|glob>
--script-args k=v        --script-timeout 30s
# categories: safe discovery auth brute vuln exploit malware default

# TIMING / EVASION
-T0..-T5 (lab: -T4)      --min-rate / --max-rate N
-f fragment              -D <decoy>,ME          --source-port 53
--spoof-mac <mac>        --data-length N        --badsum
--host-timeout 5m         --max-retries 1        --stats-every 30s

# OUTPUT
-oN <file> normal         -oX <file> XML          -oG <file> grepable
-oA <prefix> all three    --open open ports only
```

16. The professional wrap-up

Whichever exam you are preparing for, the same three habits separate a pass from a scrape:

- **Evidence over memory.** Every claim in your report traces to an `-oA` file. If you did not run it, you do not report it.
- **Permission before packets.** The lab range is yours; anything else needs written authority. State your scope in the first line of your notes.
- **Stealth awareness.** Understand what your scan looks like from the other side. The CPSA candidate uses that to choose timing; the CPIA candidate uses it to recognise an intruder in the logs.